

Register Allocation

COMS10015

Dr Tom Deakin

University of Bristol

Week 23

Register allocation

Register allocation

- ▶ In a programming language, we can declare as many variables as we like.
- ▶ However, very limited number of architectural registers.
- ▶ Some ISAs have constraints on which instructions can use which registers.
- ▶ Compiler needs to:
 - ▶ Decide which variables to store in registers.
 - ▶ Manage transfer in and out of registers.

Register spilling

- ▶ Recall: memory hierarchy.
- ▶ Accessing registers is faster than main memory.
- ▶ Program will run faster if can store most things in registers.
- ▶ If not enough registers, variables will *spill* into main memory.
- ▶ Compiler has to make good decisions of which variables to put in registers and which in memory to reduce (effect of) spilling.

Liveness analysis

- ▶ For each basic block (part of the program), calculate which variables are *live* before and after each line.
- ▶ A variable is live if is used in the future.
- ▶ I.e. It will be read before being overwritten with a new value.
- ▶ To compute liveness, walk backwards over the block, listing variables required for each line.
- ▶ If variable lifetime comes and goes, keep alive through gap.
- ▶ Liveness can be used to decide which variables to place in which registers.

Liveness analysis

$$e = d + a$$

$$f = b + c$$

$$f = f + b$$

$$d = e + f$$

$$g = d$$

Liveness analysis

$e = d + a$
 $f = b + c$
 $f = f + b$
 $d = e + f$
 $g = d$

$\{a, b, c, d\}$
 $e = d + a$
 $\{b, c, e\}$
 $f = b + c$
 $\{b, e, f\}$
 $f = f + b$
 $\{e, f\}$
 $d = e + f$
 $\{d\}$
 $g = d$
 $\{g\}$

Liveness analysis

```
e = d + a
f = b + c
f = f + b
d = e + f
g = d
```

```
{a, b, c, d}
e = d + a
{b, c, e}
f = b + c
{b, e, f}
f = f + b
{e, f}
d = e + f
{d}
g = d
{g}
```

```
{a, b, c, d}
e = d + a
{b, c, d, e}
f = b + c
{b, d, e, f}
f = f + b
{d, e, f}
d = e + f
{d}
g = d
{g}
```

Note: fill the gap to get maximum live interval for d.

Register allocation

- ▶ Variables with non-overlapping live intervals can share a register.
- ▶ Need some algorithm to allocate registers to variables live at the same time.
- ▶ Linear scan: simple greedy algorithm.
- ▶ Graph colouring: can be a better solution, but NP-complete problem with 3 or more colours¹.
 - ▶ Note: no need to compute the maximum interval with graph colouring.
 - ▶ Variables are nodes in graph, with edges when are in the same live set.
 - ▶ Colour nodes so that nodes connected by an edge have **different** colours.

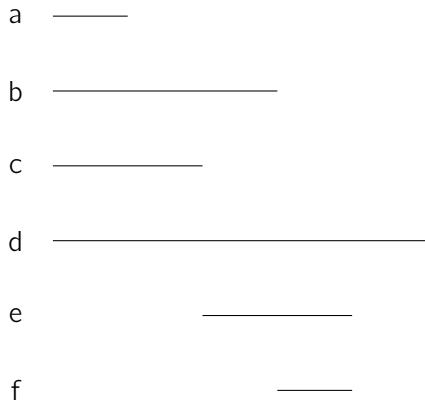
¹NP-complete problems can be verified in polynomial time, but there is no known algorithm to compute the solution in polynomial time, and can simulate other NP-complete problems.

Linear scan register allocation

- ▶ The lifetime of variables is identified by lifetime analysis.
- ▶ Variables placed in register until they are no longer needed.
- ▶ Every time a variable becomes live, need to decide where to place it.
- ▶ If there is a spare register, use it.
- ▶ If not, spill the variable to memory which has the longest time left to live.
- ▶ May evict a variable currently stored in a register.
- ▶ Linear scan is fast to compute, but might not be optimal.

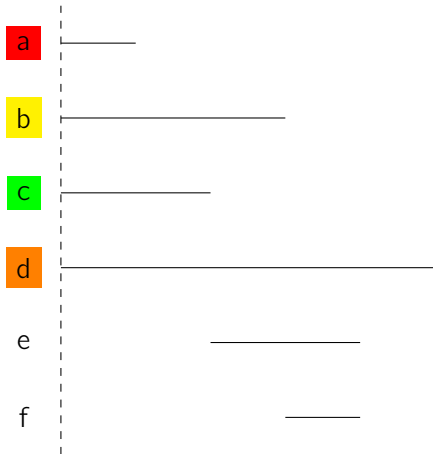
Linear scan: example

```
// Live: a, b, c, d
e = d + a;
// Live: b, c, d, e
f = b + c;
// Live: b, d, e, f
f = f + b;
// Live: d, e, f
d = e + f;
// Live: d
```



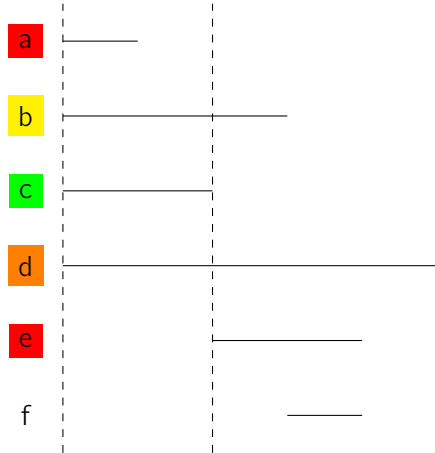
Linear scan: example

```
// Live: a, b, c, d
e = d + a;
// Live: b, c, d, e
f = b + c;
// Live: b, d, e, f
f = f + b;
// Live: d, e, f
d = e + f;
// Live: d
```



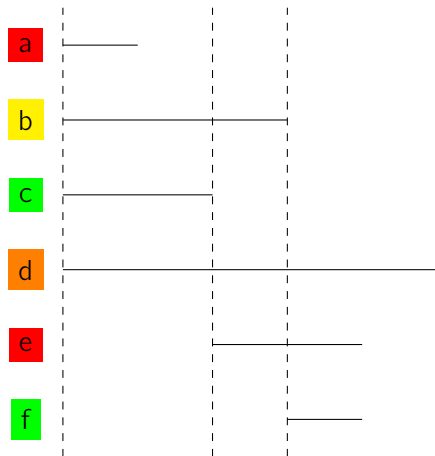
Linear scan: example

```
// Live: a, b, c, d  
e = d + a;  
// Live: b, c, d, e  
f = b + c;  
// Live: b, d, e, f  
f = f + b;  
// Live: d, e, f  
d = e + f;  
// Live: d
```



Linear scan: example

```
// Live: a, b, c, d  
e = d + a;  
// Live: b, c, d, e  
f = b + c;  
// Live: b, d, e, f  
f = f + b;  
// Live: d, e, f  
d = e + f;  
// Live: d
```

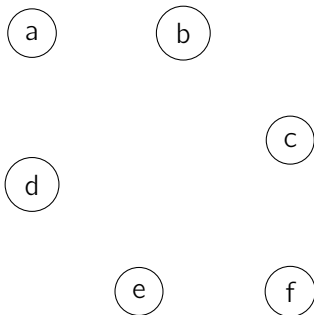


Graph coloring for register allocation

- ▶ Can construct a graph:
 - ▶ Each node is a variable.
 - ▶ Edges connect nodes where those variables are live at the same time.
- ▶ Colour the nodes so that neighbours are different colours.
 - ▶ No two nodes which are neighbours have the same colour.
- ▶ Colour shows which registers to place each variable.
- ▶ However, graph colouring is NP-hard, so approximate colouring might be used.

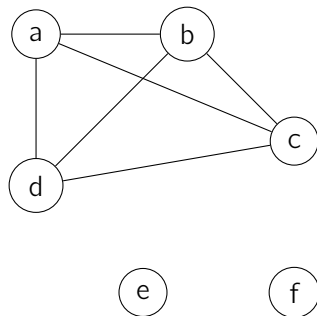
Graph colouring: example

```
e = d + a;           // Live: a, b, c, d
f = b + c;           // Live: b, c, d, e
f = f + b;           // Live: b, d, e, f
d = e + f;           // Live: d, e, f
                    // Live: d
```



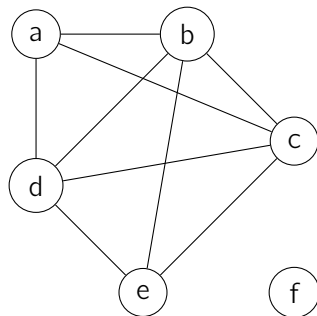
Graph colouring: example

```
e = d + a;           // Live: a, b, c, d
f = b + c;           // Live: b, c, d, e
f = f + b;           // Live: b, d, e, f
d = e + f;           // Live: d, e, f
                     // Live: d
```



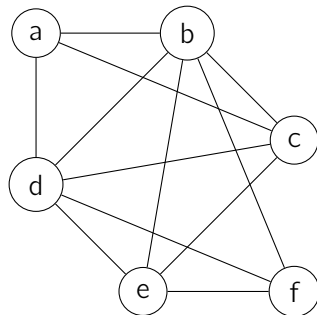
Graph colouring: example

```
e = d + a;           // Live: a, b, c, d
f = b + c;           // Live: b, c, d, e
f = f + b;           // Live: b, d, e, f
d = e + f;           // Live: d, e, f
                     // Live: d
```



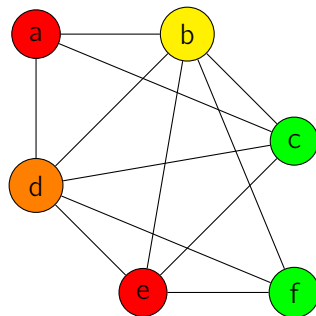
Graph colouring: example

```
e = d + a;           // Live: a, b, c, d
f = b + c;           // Live: b, c, d, e
f = f + b;           // Live: b, d, e, f
d = e + f;           // Live: d, e, f
                     // Live: d
```



Graph colouring: example

```
e = d + a;           // Live: a, b, c, d
f = b + c;           // Live: b, c, d, e
f = f + b;           // Live: b, d, e, f
d = e + f;           // Live: d, e, f
                    // Live: d
```



Four registers (colours)

Further Reading

- ▶ https://en.wikipedia.org/wiki/Register_allocation
- ▶ <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/17/Slides17.pdf>
- ▶ <https://www.inf.ed.ac.uk/teaching/courses/copt/lecture-7.pdf>
- ▶ <http://www.cs.cmu.edu/afs/cs/academic/class/15745-s13/public/lectures/L15-Register-Allocation.pdf>
- ▶ https://en.wikipedia.org/wiki/Live_variable_analysis
- ▶ <http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf>